

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..32

SINGLE-PAGE APPS

 Copy as Markdown

Single-page apps

Single-page applications (SPAs) handle routing on the client side, which requires special server configuration. When users navigate to a route like `/dashboard` or `/settings`, the browser requests that path from the server. Since these aren't real files, the server needs to return the main `index.html` file so the client-side router can handle the route.

Serverpod provides `SpaRoute` to handle this pattern automatically.

Basic setup

Use `SpaRoute` to serve your SPA with automatic fallback to `index.html`:

```
final webDir = Directory('web/app');

pod.webServer.addRoute(
  SpaRoute(
    webDir,
    fallback: File('web/app/index.html'),
  ),
);
```

INFO

The route path defaults to `'/'` (root). See [Serving from a sub-path](#) to mount the SPA at a different location.

This configuration:

- Serves static files from `web/app` when they exist
- Falls back to `index.html` for any path that doesn't match a file



- Enables client-side routing frameworks (React Router, Vue Router, etc.) to work correctly

How it works

When a request comes in:

1. The `SpaRoute` first tries to serve a matching static file from the directory
2. If no file exists (404 response), it serves the fallback file instead
3. The client-side JavaScript then handles routing based on the URL

This is implemented using `FallbackMiddleware` internally, which you can also use directly for custom fallback behavior.

Cache control

Configure caching for your static assets:

```
pod.webServer.addRoute(  
  SpaRoute(  
    webDir,  
    fallback: File('web/app/index.html'),  
    cacheControlFactory: StaticRoute.public(  
      maxAge: const Duration(minutes: 5),  
    ),  
  ),  
);
```

To set the header per environment instead of in code, set `SERVERPOD_WEB_SERVER_SPA_CACHE_CONTROL` to the `Cache-Control` value the route should send:

```
SERVERPOD_WEB_SERVER_SPA_CACHE_CONTROL="public, max-age=3600"
```

The environment variable applies to every `SpaRoute` that does not pass a `cacheControlFactory` of its own. Like the factory, it never applies to the fallback file, which is always served without a `Cache-Control` header so a deploy is never masked by a cached app shell. The `SERVERPOD_WEB_SERVER_STATIC_CACHE_CONTROL` variable used by `StaticRoute` does not apply here.

See [Static files](#) for more on cache control, including why `publicImmutable` belongs only with cache-busted assets.

Cache busting

Enable cache-busted URLs for your assets:

```
final webDir = Directory('web/app');

final cacheBustingConfig = CacheBustingConfig(
  mountPrefix: '/',
  fileSystemRoot: webDir,
);

pod.webServer.addRoute(
  SpaRoute(
    webDir,
    fallback: File('web/app/index.html'),
    cacheBustingConfig: cacheBustingConfig,
    cacheControlFactory: StaticRoute.publicImmutable(
      maxAge: const Duration(days: 365),
    ),
  ),
);
```

See [Static files](#) for more on cache busting.

Using FallbackMiddleware directly

For more control, use `FallbackMiddleware` with `StaticRoute`:

```
final webDir = Directory('web/app');
final indexFile = File('web/app/index.html');

pod.webServer.addMiddleware(
  FallbackMiddleware(
    fallback: StaticRoute.file(indexFile),
    on: (response) => response.statusCode == 404,
  ).call,
  '/',
);

pod.webServer.addRoute(StaticRoute.directory(webDir), '/');
```

This gives you flexibility to customize the fallback condition. For example, you could fall back on any 4xx error:

```
FallbackMiddleware(
  fallback: StaticRoute.file(indexFile),
  on: (response) => response.statusCode ≥ 400 && response.statusCode < 500,
).call
```

Serving from a sub-path

To serve your SPA from a sub-path instead of the root, pass the path as the second argument to `addRoute`:

```
final webDir = Directory('web/app');

pod.webServer.addRoute(
  SpaRoute(
    webDir,
    fallback: File('web/app/index.html'),
  ),
  '/app',
);
```

This serves the SPA at `/app`, so users would access it at `http://localhost:8082/app`.

NOTE

When using cache busting with a sub-path, update the `mountPrefix` to match:

```
final cacheBustingConfig = CacheBustingConfig(
  mountPrefix: '/app',
  fileSystemRoot: webDir,
);
```

Serving Flutter web applications

For serving Flutter web applications specifically, see [Flutter web apps](#).